Hardware-to-Simulation Methodology Notes on CAD-Based Parameter Identification and Isaac Lab Actuator Calibration

Repo `exo-actuator-sensor-drivers`

May 2026

Key Result

This note summarizes a practical sim-to-real methodology adapted from the video transcript provided by the user. The main idea is:

1. build a physically credible URDF from measured hardware and CAD,
2. tune the actuator model in Isaac Lab against real actuator data,
3. only then train or evaluate policies for transfer to hardware.

Contents

1 Overview

The methodology has three coupled parts:

1. identify the rigid-body parameters used by the URDF,
2. calibrate the simulated actuator model against real actuator response,
3. close the remaining sim-to-real gap at the sensing and runtime level.

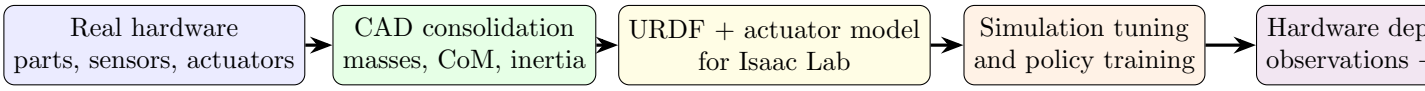

Figure 1: Overall methodology: accurate rigid-body parameters and actuator calibration are both needed before policy transfer to hardware is credible.

2 Rigid-Body Parameter Identification from Hardware and CAD

The key rigid-body quantities that must appear in the URDF are:

- mass of each link or rigid subassembly,
- centre-of-mass location of each rigid subassembly,
- moment of inertia tensor of each rigid subassembly,
- joint locations and joint axes.

The important methodological point is that CAD is not used as a purely nominal geometric model. Instead, CAD is adjusted so that each rigid body matches the real hardware mass as closely as possible.

2.1 What comes from physical parts, Fusion, and actuator modeling?

For computed-torque modeling, it helps to separate the parameter sources explicitly. The table below summarizes a practical split.

Quantity / group	From physical parts	From Fusion / CAD	Role in CT / dynamics model
Link masses m_i	Weigh each rigid assembly with screws, brackets, wires, electronics, and mounted hardware included	Use CAD only to organize the assembly and verify consistency	Enters rigid-body dynamics directly; affects $\boldsymbol{M}(\boldsymbol{q})$, $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$, and $\boldsymbol{g}(\boldsymbol{q})$
Centre of mass l_{ci} or full CoM position	Usually hard to measure accurately on the finished robot	Extract from the corrected CAD assembly after masses are matched	Needed for gravity and coupling terms; strongly affects $\boldsymbol{g}(\boldsymbol{q})$ and also $\boldsymbol{M}, \boldsymbol{C}$
Moments / tensor of inertia I_i	Not usually measured directly for each link in practice	Extract from Fusion / CAD after the link mass properties are corrected	Needed mainly for $\boldsymbol{M}(\boldsymbol{q})$ and indirectly for $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$
Joint axes and joint locations	Measurable geometrically, but tedious and error-prone	Best taken from CAD / URDF geometry	Defines the kinematics that the CT model is built on
Base and link geometry	Physical dimensions can be checked with calipers	Usually taken from CAD model and exported into URDF	Supports kinematics, collision, and visualization; also affects inertial calculations
Rotor / reflected actuator inertia	Sometimes estimated from datasheet or special experiments	Can be represented separately in the actuator model or reflected into the joint model	Important if actuator dynamics are not negligible compared with link inertia
Motor bandwidth, delay, acceleration limits, inner-loop behavior	Identify from real actuator tests using encoder traces	Not obtained from rigid CAD	Belongs to actuator modeling, not the pure rigid-body CT model; needed so simulation torque/position response matches hardware
Friction, damping, backlash, deadzone	Identified from experiments on the real robot	Not usually available from CAD alone	Often added as extra non-ideal terms beyond the textbook $\boldsymbol{M}\ddot{\boldsymbol{q}} + \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}$ model

Table 1: Practical source split for computed-torque modeling. Physical measurements give masses and validation data; Fusion / CAD gives CoM and inertia information after the model is mass-corrected; actuator behavior must be identified separately from dynamic response tests.

Key Result

For computed torque, the rigid-body part of the model is usually built from measured masses plus Fusion/CAD-derived CoM and inertia data. The actuator is then modeled separately using response calibration, because its bandwidth, delay, and inner-loop behavior do not come from the rigid URDF alone.

2.2 Part-by-Part Mass Matching

The methodology described in the transcript is practical and defensible:

1. Split the robot into rigid assemblies that will appear in the URDF.
2. Weigh the real printed parts one by one.
3. Include attached screws, brackets, electronics, wires, capacitors, and other mounted items in the measured mass of the corresponding assembly.
4. In CAD, assign material or density values so that each CAD body reproduces the measured real mass.

5. Combine the bodies that belong to the same URDF link and extract the total mass, centre of mass, and moment of inertia.

Important

The important lesson is that a geometrically correct CAD model is not enough. If the real printed link contains screws, rotor hardware, wiring, or attached electronics, those items must be accounted for in the mass properties used by the URDF.

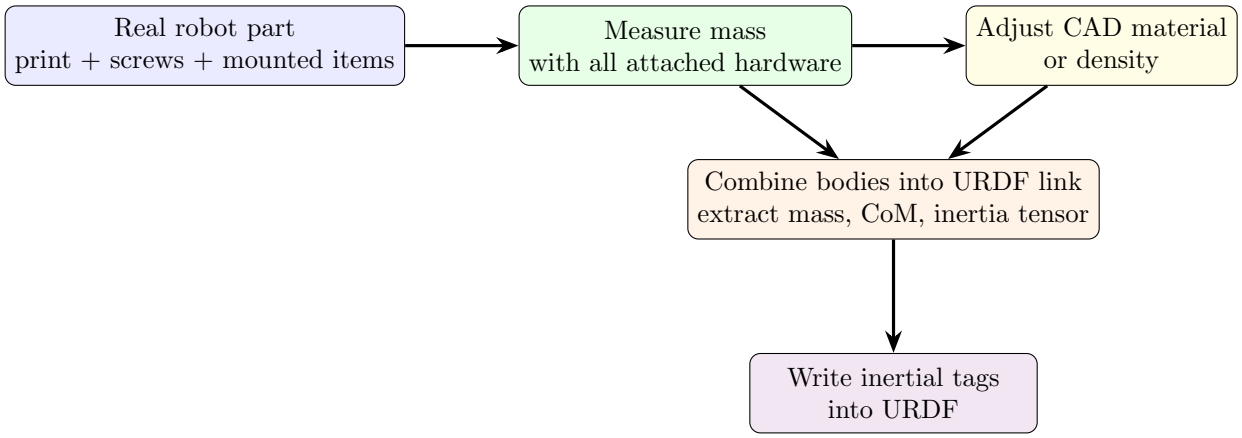

Figure 2: Hardware-to-CAD parameter flow. The CAD model is used as an inertial bookkeeping tool after its mass properties are aligned with the real robot.

2.3 Rotor and Actuator Separation

The transcript also highlights a more refined modeling step: separate the actuator housing from the rotor where possible. This is useful because the rotor inertia may belong dynamically to a different side of the transmission than the stator housing. Even if the final URDF uses a simplified rigid-link representation, thinking in terms of rotor-side and body-side inertia helps avoid hiding major inertia in the wrong link.

3 Actuator Model Calibration for Isaac Lab

Once the URDF has credible rigid-body parameters, the next problem is actuator fidelity. A robot can still transfer poorly if the simulated actuator dynamics do not match the real actuators.

The methodology in the transcript is straightforward:

1. implement the same actuator-side behavior in simulation that is present on the real hardware,
2. command the real actuator with simple test signals,
3. run the same signals in simulation,
4. compare encoder traces,
5. tune actuator parameters until the responses are close.

In the video, this includes adding acceleration-related behavior to the simulated actuator model because the real actuator clearly behaves differently with and without that effect.

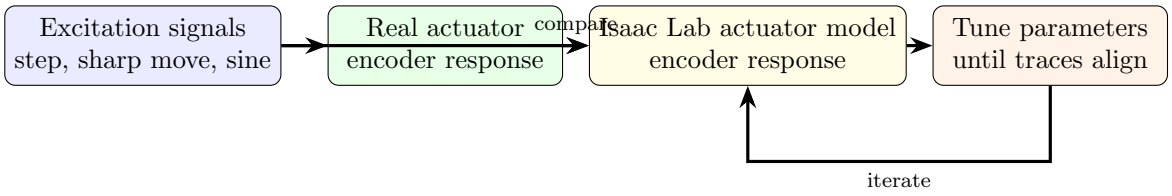

Figure 3: Actuator calibration loop: excite the real and simulated actuators with the same commands, compare encoder traces, and iterate on model parameters.

3.1 Transcript-Based Identification Process: Sharp Command and Sine Wave

The transcript describes a practical identification loop rather than a purely analytical derivation. The process is:

1. choose one actuator at a time,
2. command the real actuator with a *sharp* move and with a sinusoidal trajectory,
3. apply the same commands to the simulated actuator model,
4. log encoder response from both systems,
5. tune the simulated actuator parameters until the traces are close.

In the transcript, the important modeling decision was to add **acceleration-related behavior** to the simulated actuator because the real actuator clearly behaved differently with and without that effect.

3.1.1 Step 1: Use a sharp command to identify transient behavior

The transcript's "sharp command" is best interpreted as a fast setpoint change or a high-jerk input used to expose transient effects. This test is useful for identifying:

- command delay or dead time,
- rise time,
- acceleration limiting,
- overshoot,
- effective damping,
- saturation or clipping in the inner loop.

A useful low-order simulated position-actuator model is:

$$\ddot{q} = \mathrm{sat}_{a_{\max}}\left(\omega_n^2\left(q_{\mathrm{cmd}} - q\right) - 2\zeta\omega_n\,\dot{q}\right), \tag{1}$$

where ω_n is the effective bandwidth, ζ is the damping ratio, and $a_{\max}$ is an acceleration limit.

For a sharp command, the measured encoder trace is compared against the simulated response to fit:

- $a_{\max}$ from the initial slope and curvature,
- ω_n from how quickly the response rises,
- ζ from overshoot and settling behavior,
- explicit delay if the real actuator starts moving later than the model.

3.1.2 Step 2: Use a sine wave to identify frequency response

Once the transient response is roughly correct, the transcript's sinusoidal test is used to check whether the actuator tracks periodic motion with the correct amplitude and phase.

With a commanded signal

$$q_{\mathrm{cmd}}(t) = A\sin(\omega t), \tag{2}$$

the measured actuator response can be approximated as

$$q(t) \approx B(\omega)\sin\big(\omega t - \phi(\omega)\big), \tag{3}$$

where $B(\omega)$ is the output amplitude and $\phi(\omega)$ is the phase lag.

This test identifies:

- actuator bandwidth,
- phase lag,
- high-frequency attenuation,
- lightly damped resonance or compliance effects.

In practice, one increases sine frequency gradually and checks whether the simulated model loses amplitude and accumulates phase lag in the same way as the real actuator.

3.1.3 Step 3: Compare encoder traces and iterate

The transcript's comparison variable is the encoder trace itself. This is a good practical choice because it avoids arguing about hidden internal states: the only requirement is that the simulated actuator and the real actuator show similar externally visible motion under the same test command.

The fitting loop is therefore:

1. run a sharp command on the real actuator and log encoder position,
2. run the same command in simulation and log simulated position,
3. adjust delay, acceleration limit, damping, and bandwidth,
4. run a sine-wave command at one or more frequencies,
5. adjust the model until amplitude and phase match acceptably,
6. repeat actuator by actuator.

Important

This procedure identifies the *effective closed-loop actuator behavior*, not necessarily the internal motor physics in detail. For direct-drive joints that may be sufficient. For cable-driven or SEA joints, the fitted response can include motor dynamics, transmission compliance, cable damping, and pulley effects all together.

3.1.4 Step 4: Apply the same idea to our planar cable-driven arm

For our own planar robot, the same transcript methodology should be applied joint by joint:

- **Shoulder (AK80-kv60):** use sharp commands and sine waves to fit the effective direct-drive actuator response.
- **Elbow (AK60-kv80 with steel cables):** use the same tests, but interpret the identified model as actuator-plus-transmission behavior, because the measured encoder response is influenced by cable stretch, damping, pulley geometry, and any SEA element in the drive path.

For the elbow, a simple transmission-side quantity is the cable extension

$$\delta = r_p(\theta_m - q_2), \tag{4}$$

which can be used in a compliant drive model such as

$$F = k_s\,\delta + b_c\,\dot{\delta}, \qquad \tau_2 = r_p F. \tag{5}$$

The sharp and sine tests on the elbow then calibrate not only the motor-side bandwidth, but also the effective compliance seen at the joint.

3.2 Why This Calibration Matters

If the rigid-body model is good but the actuator is too ideal in simulation, the learned policy will exploit actuator behavior that does not exist on the real robot. Conversely, if the actuator model is close but the URDF inertial properties are poor, the policy will still see the wrong dynamics. Both must be addressed together.

4 Deployment Constraints Beyond the Physics Model

The transcript also makes an important point that is easy to underestimate: sim-to-real transfer is not only a physics problem. It is also an observation, estimation, and timing problem.

4.1 Observation Availability

The policy may require quantities such as:

- joint positions and velocities,
- commanded body velocities,
- projected gravity from the IMU,
- base angular velocity,
- base linear velocity,
- previous action.

In the transcript, base linear velocity becomes the main obstacle because it is not directly available from simple sensors and normally requires a state estimator. The practical workaround described there is to use a SLAM camera to obtain pose and velocity signals directly, avoiding a full estimator in the first iteration.

4.2 Timing and Compute Path

Even with a good model and a trained policy, deployment can fail if the sensing and actuation loop cannot meet the required update rate. The transcript gives an example of a 50 Hz policy loop, implying a full sensor-read, policy, and motor-command cycle of less than 20 ms.

This turns transport and compute hardware into part of the methodology:

- CAN transport latency matters,
- loop-time consistency matters, not just mean latency,
- embedded compute choice changes inference and I/O jitter.

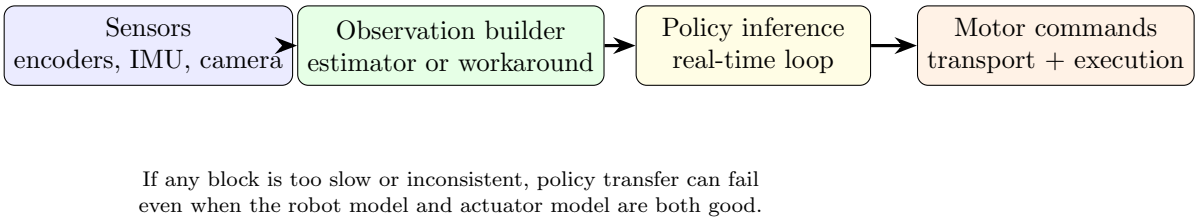

Figure 4: Deployment path for policy transfer. Physics fidelity is necessary, but observation availability and timing consistency are also first-order constraints.

5 Methodology Adapted to Our Workflow

For our own robotics workflow, the methodology can be written as:

1. Measure each real rigid assembly carefully, including fasteners and attached components.
2. Use CAD as an inertial bookkeeping tool, not just a geometry export tool.
3. Populate URDF inertial tags from the corrected CAD model.
4. Build or tune actuator dynamics in Isaac Lab or Isaac Sim using real actuator response data.
5. Verify that the simulated actuator reproduces step and sinusoidal behavior seen on the real robot.
6. Check whether the deployed policy's required observations are actually available on the hardware stack.
7. Validate that the runtime platform can sustain the required control-loop period with acceptable jitter.

Key Result

The main methodological lesson is that sim-to-real success is not obtained by "URDF only" or "policy only". It comes from three aligned layers: accurate rigid-body parameters, calibrated actuator dynamics, and a hardware runtime stack that can supply the required observations at the needed rate.